



Kth Largest Number in a Stream (medium)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
- Code
 - Time complexity
 - Space complexity

Problem Statement

Design a class to efficiently find the Kth largest element in a stream of numbers.

The class should have the following two things:

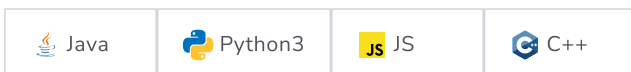
1. The constructor of the class should accept an integer array containing initial numbers from the stream and an integer 'K'.
2. The class should expose a function `add(int num)` which will store the given number and return the Kth largest number.

Example 1:

Input: [3, 1, 5, 12, 2, 11], K = 4
1. Calling `add(6)` should return '5'.
2. Calling `add(13)` should return '6'.
2. Calling `add(4)` should still return '6'.

Try it yourself

Try solving this question here:



```
1 import java.util.*;
2
3 class KthLargestNumberInStream {
4
5     public KthLargestNumberInStream(int[] nums, int k) {
6         // TODO: Write your code here
7     }
8
9     public int add(int num) {
10        // TODO: Write your code here
```



```

11     return -1;
12 }
13
14 public static void main(String[] args) {
15     int[] input = new int[] { 3, 1, 5, 12, 2, 11 };
16     KthLargestNumberInStream kthLargestNumber = new KthLargestNumberInStream(input, 4);
17     System.out.println("4th largest number is: " + kthLargestNumber.add(6));
18     System.out.println("4th largest number is: " + kthLargestNumber.add(13));
19     System.out.println("4th largest number is: " + kthLargestNumber.add(4));
20 }
21 }

```

Run

Save

Reset



Solution

This problem follows the **Top 'K' Elements** pattern and shares similarities with [Kth Smallest number](#).

We can follow the same approach as discussed in the 'Kth Smallest number' problem. However, we will use a **Min Heap** (instead of a **Max Heap**) as we need to find the Kth largest number.

Code

Here is what our algorithm will look like:

Java

Python3

C++

JS

```

1  import java.util.*;
2
3  class KthLargestNumberInStream {
4      PriorityQueue<Integer> minHeap = new PriorityQueue<Integer>((n1, n2) -> n1 - n2);
5      final int k;
6
7      public KthLargestNumberInStream(int[] nums, int k) {
8          this.k = k;
9          // add the numbers in the min heap
10         for (int i = 0; i < nums.length; i++)
11             add(nums[i]);
12     }
13
14     public int add(int num) {
15         // add the new number in the min heap
16         minHeap.add(num);
17
18         // if heap has more than 'k' numbers, remove one number
19         if (minHeap.size() > this.k)
20             minHeap.poll();
21
22         // return the 'Kth largest number
23         return minHeap.peek();
24     }
25
26     public static void main(String[] args) {
27         int[] input = new int[] { 3, 1, 5, 12, 2, 11 };
28         KthLargestNumberInStream kthLargestNumber = new KthLargestNumberInStream(input, 4);
29         System.out.println("4th largest number is: " + kthLargestNumber.add(6));
30         System.out.println("4th largest number is: " + kthLargestNumber.add(13));
31         System.out.println("4th largest number is: " + kthLargestNumber.add(4));
32     }
33 }

```



Run

Save

Reset



Time complexity



The time complexity of the `add()` function will be $O(\log K)$ since we are inserting the new number in the heap.

Space complexity

The space complexity will be $O(K)$ for storing numbers in the heap.

[< Back](#)[✓ Mark As Completed](#)[Next >](#)[Frequency Sort \(medium\)](#)['K' Closest Numbers \(medium\)](#)